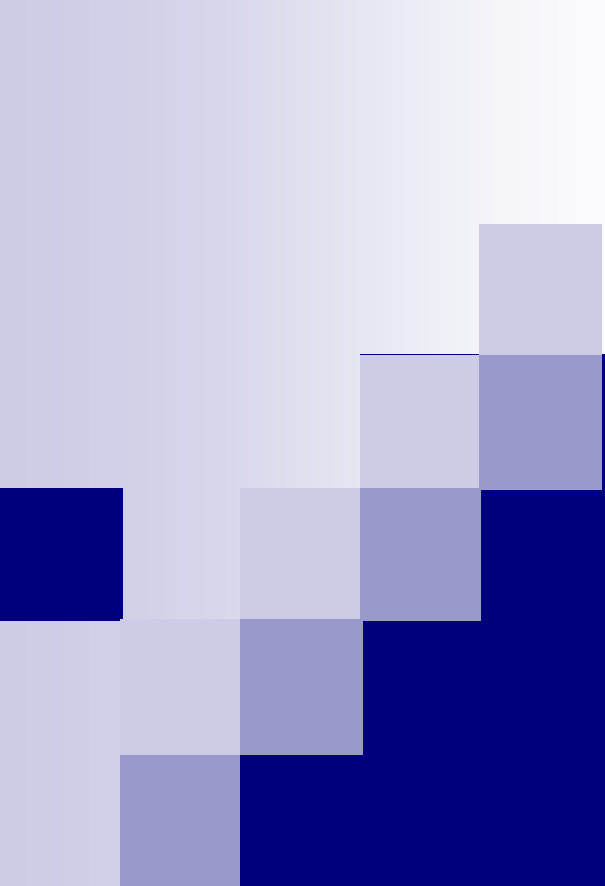




Networks

Subhankar Jana
Btech, 3rd Sem



CONTENTS

- Network flows on directed acyclic graphs
- Ford-fulkerson Algorithms
 - Residual networks

Network flows on directed acyclic graphs

- DAG is a directed graph with no cycles.
- Flow network: a flow network $G = (V, E)$ is a directed graph in which each edge $(u, v) \in E$ has a nonnegative capacity $c(u, v) \geq 0$. if $(u, v) \notin E$, we assume that $c(u, v) = 0$.

We distinguish two vertices in flow network, source s and sink t .

- The source produces the material at a steady rate.
- The sink consumes the material at a steady rate.

Objective : how much of material can be imposed through the network that it can handle.

- Flow: A flow in G is a real-valued function $f : V \times V \rightarrow \mathbb{R}$ that satisfies three properties:
- 1. Capacity constraint : For all $u, v \in V$, we require $f(u, v) \leq c(u, v)$.
- The net flow from one vertex to another must not exceed the given capacity.
- 2. Flow conservation : For all $u \in V - \{s, t\}$, we require

$$\sum_{v \in V} f(u, v) = 0.$$

The quantity $f(u, v)$, which can be positive or negative, is called the net flow from vertex u to v .

The value of the flow is defined as $|f| = \sum_{v \in V} f(s, v)$ that is the total net

Flow out of the source.

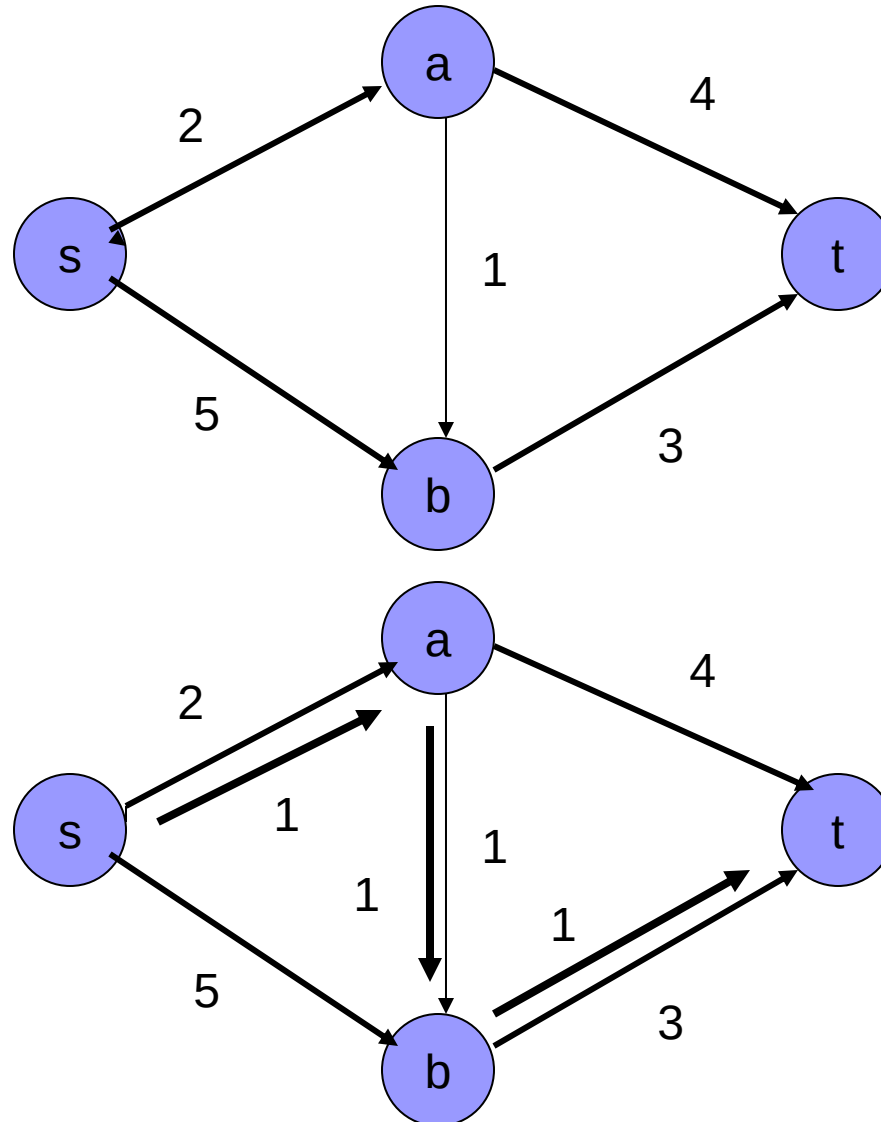
Residual Capacity

- Given a flow f in network $G = (V, E)$.
- Consider a pair of vertices $u, v \in V$.
- **Residual capacity:**
 - The amount of additional flow we can push directly from u to v .
 - $c_f(u, v) = c(u, v) - f(u, v)$
 ≥ 0 (since $f(u, v) \leq c(u, v)$)
- Example:
 - $c(u, v) = 16, f(u, v) = 5 \rightarrow c_f(u, v) = 11$

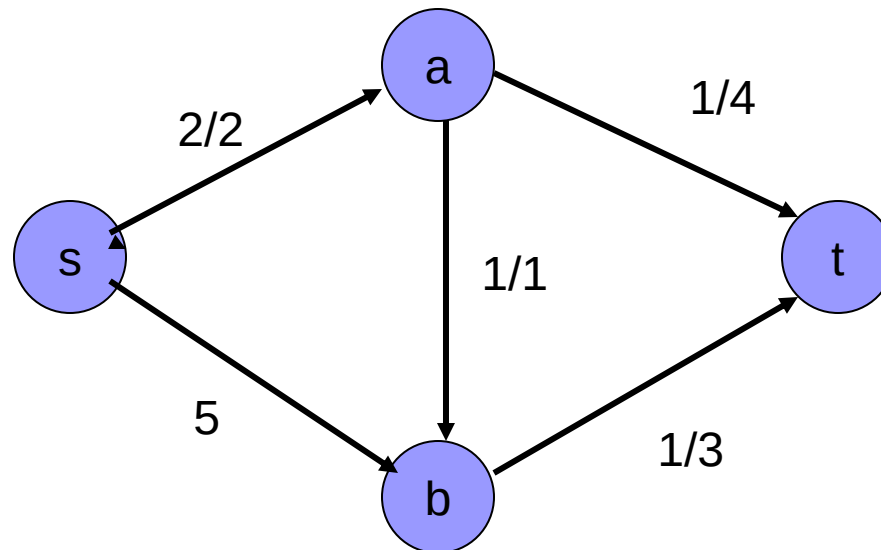
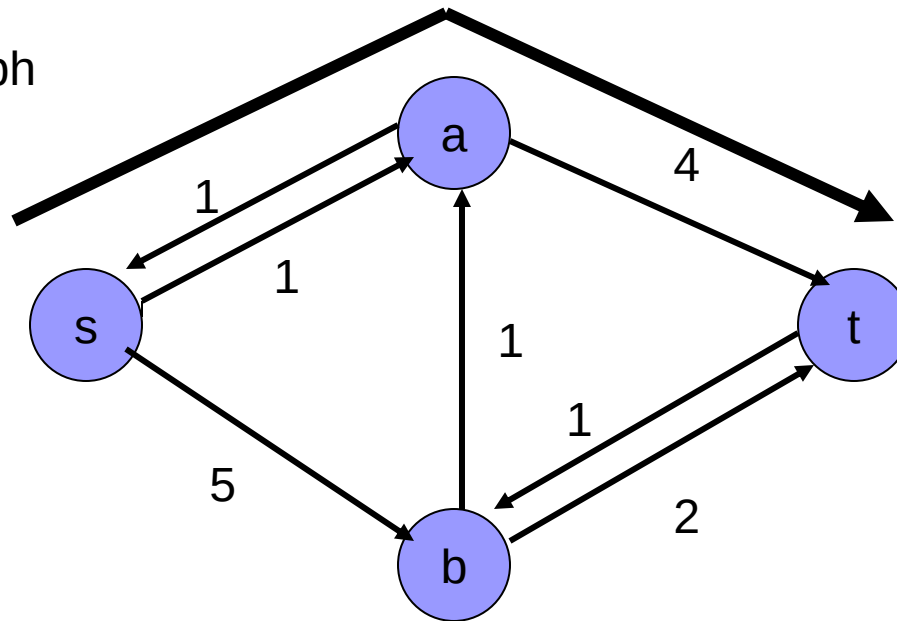
Residual Network

- **Residual network:** $G_f = (V, E_f)$,
 $E_f = \{(u, v) \in V \times V : c_f(u, v) > 0\}$.
- Each edge of the residual network can admit a positive flow.
- Given flows f_1 and f_2 , the **flow sum** $f_1 + f_2$ is
 - $(f_1 + f_2)(u, v) = f_1(u, v) + f_2(u, v)$
for all $u, v \in V$.

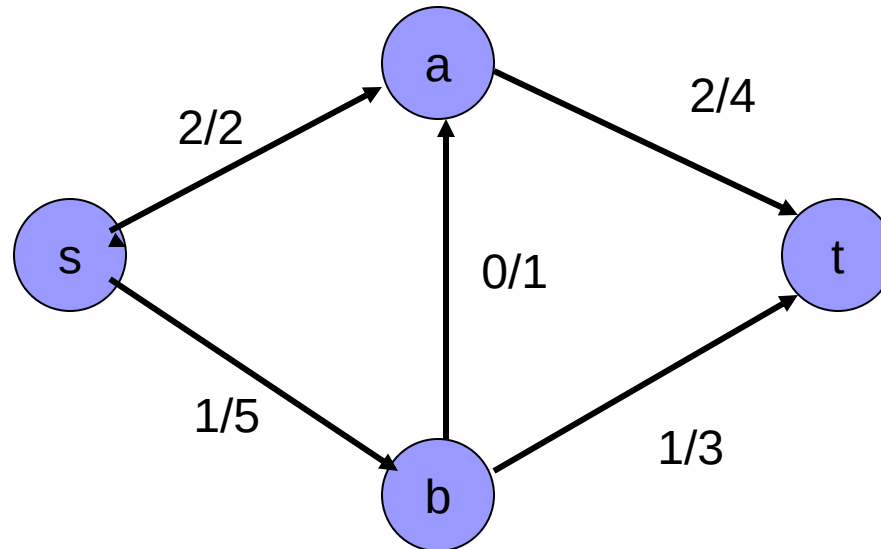
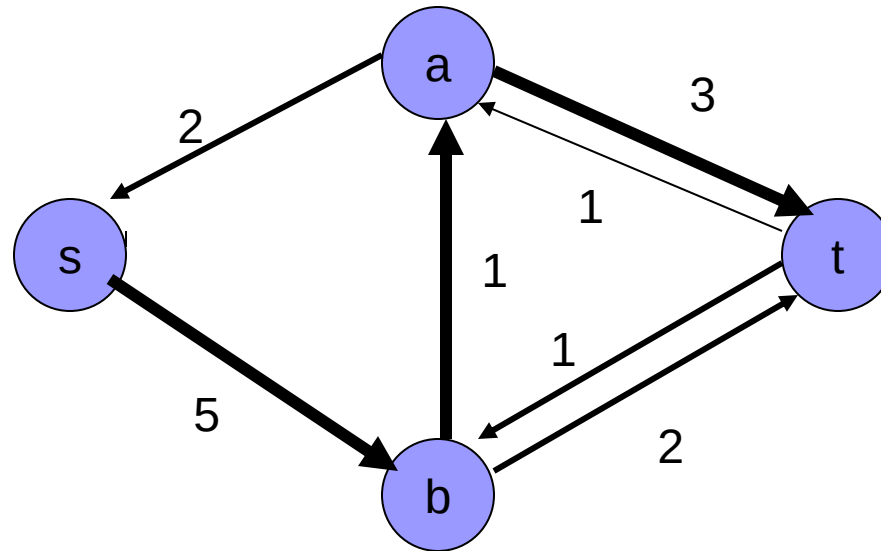
Consider the network $G=(V,E)$ shown in the figure below. Each edge $(u,v) \in E$ in the network is labeled with its capacity $c(u,v)$.

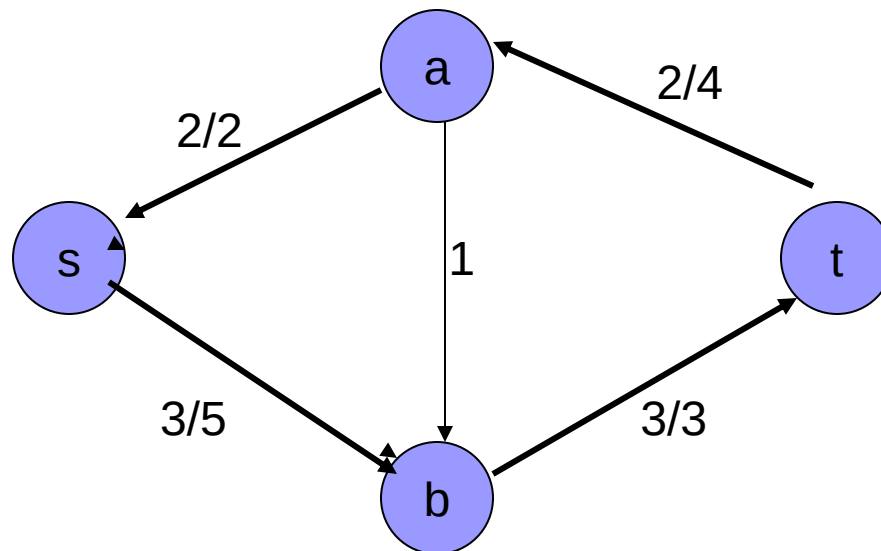
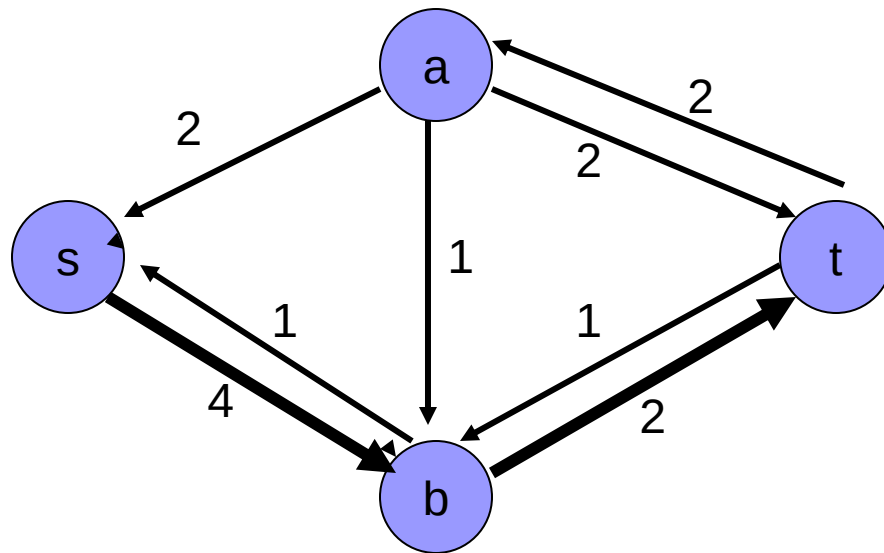


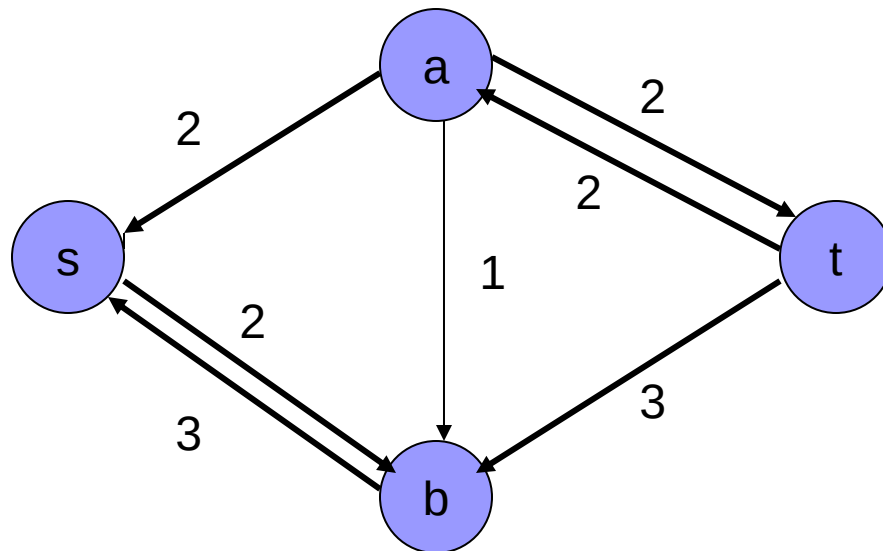
Residual graph



Residual graph







Max Flow : 5

Ford-Fulkerson Algorithm

- The Ford_Fulkerson method is iterative,
- Starts with $f(u,v)$ for $(u,v) \in V$, initial flow of value 0.
- The method is based on the augmenting path which is
- defined as a path from s to t along which we can push
- more flow and then augment flow along this path.



■ Procedure Ford_Fulkerson_method(G, s, t): For each edge $(u, v) \in E[G]$

do $f[u, v] \leftarrow 0$

$f[v, u] \leftarrow 0$

While there exists a path p from s to t in the residual network G_1

do $c_1(p) \leftarrow \min\{c_1(u, v) : (u, v) \text{ is in } p\}$

for each edge (u, v) in p

do $f[u, v] \leftarrow f[u, v] + c_1(p)$

$f[v, u] \leftarrow -f[u, v]$

Augmenting Paths in a Residual Graph

■ Now we want to make precise the way in which we push flow from s to t in G_f . Let P be a simple s - t path in G_f —that is, P does not visit any node more than once.

We define $\text{bottleneck}(P, f)$ to be the minimum residual capacity of any edge on P , with respect to the flow f .

We now define the following operation $\text{augment}(f, P)$, which yields a new flow f' in G

Augment(f , P)

Let $b = \text{bottleneck}(P, f)$

For each edge $(u, v) \in P$

 If $e = (u, v)$ is a forward edge then increase $f(e)$ in G by b

 Else $((u, v)$ is a backward edge, and let $e = (v, u)$) decrease $f(e)$ in G by b

 End if

End for

Return(f)

Max-Flow

Initially $f(e) = 0$ for all e in G

While there is an s-t path in the residual graph G_f

Let P be a simple s-t path in G_f

$f' = \text{augment}(f, P)$

Update f to be f'

Update the residual graph G_f to be $G_{f'}$

End while

Return f

Example:

